

Parte 4 - Sviluppo DotNET

Introduzione a C#

- **Definizione:** C# è un linguaggio di programmazione general-purpose (adatto a qualsiasi generi di software), a differenza dei linguaggi domain-specific (come SQL o PHP).
- **Storia:** sviluppato da Microsoft a partire dal 2000, ispirato a Java e C++ e legato allo sviluppo di .NET Framework.
- **Caratteristiche principali:**
 - Strongly-typed (fortemente tipizzato).
 - Orientato agli oggetti (OOP).
 - Gestito tramite garbage collector.
 - Open Source e Cross-platform.

Il sistema dei tipi (strongly-typed)

- **Differenza con linguaggi dinamici:** a differenza di Python (weakly-typed), dove una variabile può cambiare tipo liberamente, in C# il tipo è vincolante.
- **Controllo dei tipi:** se si dichiara un `int`, non è possibile assegnargli una stringa; il compilatore genera l'errore "cannot implicitly convert type 'string' to 'int'".

```
test.py
1 # Python 3 (weakly-typed)
2 x = 1
3 print(f"x è {x}")
4
5 x = "Hello"
6 print(f"x è {x}")

Output:
x è 1
x è Hello

Program.cs
1 // C# (strongly-typed)
2 int x = 1;
3 Console.WriteLine($"x è {x}");
4
5 x = "Hello"; Cannot implicitly convert type 'string' to 'int'
6 Console.WriteLine($"x è {x}");
```

Programmazione orientata agli oggetti (OOP)

C# si basa su classi, oggetti, ereditarietà e polimorfismo.

Classi e membri

- **Classi:** definiscono la struttura degli oggetti (es. `class Cane`). possono contenere costruttori per l'inizializzazione.
- **Atributi (Campi vs. Proprietà):**
 - **Campi:** variabili semplici all'interno della classe (es. `public string Campo;`).
 - **Proprietà:** forniscono metodi *getter* e *setter* impliciti (es. `public string Proprietà { get; set; }`). Sono più flessibili e generalmente preferibili rispetto all'esposizione diretta dei campi.

- **Metodi:** definiscono i comportamenti (es. `Abbaia()` , `CambiaNome()`).

Ereditarietà e polimorfismo

- **Ereditarietà:** una classe può ereditare da una classe base (es. `class Cane : Animale`). Le sottoclassi usano `override` per implementare metodi astratti della classe base.
- **Polimorfismo:** è possibile trattare oggetti di tipo diverso (es. `Cane` e `Gatto`) tramite la loro classe base comune (`Animale`), invocando metodi che si comportano diversamente in base all'implementazione specifica.

Interfacce

- **Definizione:** definiscono un contratto (metodi/proprietà) senza contenere dati.
- **Differenza con le classi:** una classe può avere una sola classe base ma implementare più interfacce.

Architettura ed esecuzione

- **Compilazione:**
 1. Sorgenti (`.cs`) compilati (es. via Roslyn) in **Assembly** (`.dll`).
 2. Gli assembly contengono **bytecode** (CIL).
- **Runtime:** il codice viene eseguito da una virtual machine (CLR) che utilizza un compilatore JIT (Just-In-Time) per tradurre il bytecode in codice macchina.
- **Gestione memoria:** la memoria è gestita implicitamente (niente `malloc` / `free`). Un garbage collector libera automaticamente la memoria non più utilizzata.

Ecosistema .NET

- **Evoluzione Open Source:** si è passati dal .NET Framework (proprietario) a .NET Core e .NET (open source).
- **Cross-platform:** il runtime supporta Windows, Linux, Mac, Android e iOS (tramite Mono/Xamarin) e diverse architetture (x86, ARM).
- **Strumenti:** visual studio (Windows), Visual Studio Code (Multiplatforma), ILSpy, SharpLab.

Modificatori di visibilità

- **public:** visibile ovunque.
- **private:** visibile solo nella stessa classe (default per i membri interni).
- **protected:** visibile nella classe e nelle sottoclassi.
- **internal:** visibile ovunque nello stesso assembly (default per le classi).
- *Nota:* nelle interfacce, i membri sono `public` di default.